

Key-Value Store v8

Versioni e compare-and-set

Architetture dei Sistemi Distribuiti

Lezione 17

Perche' Non Basta SET

Finora il client poteva fare due cose:

- leggere un valore;
- sovrascriverlo con SET.

Ma SET e' una *blind write*:

- non dice quale stato il client avesse osservato;
- non protegge da aggiornamenti concorrenti;
- non distingue “scrittura legittima” da “scrittura su stato ormai vecchio”.

La domanda di fondo e':

il client puo' ancora fidarsi dello stato che aveva letto prima di scrivere?

Se la risposta e' "forse no", serve un contratto piu' ricco:

- il server deve esporre uno stato osservato;
- il client deve poter subordinare la scrittura a quello stato;
- il fallimento deve essere semantico, non solo tecnico.

La lezione introduce due comandi:

- GETV <key>: restituisce valore e versione;
- CAS <key> <expected_version> <value>: scrive solo se la versione e' quella attesa.

Da questo momento la versione:

- non e' piu' solo metadato interno;
- entra nel linguaggio del client;
- diventa parte del contratto dell'API.

```
SET course ads  
-> OK version=0
```

```
GETV course  
-> OK ads version=0
```

```
CAS course 0 advanced-distributed-systems  
-> OK version=1
```

Interpretazione:

- il client ha osservato la versione 0;
- il server conferma che nulla e' cambiato nel frattempo;
- la scrittura viene accettata e la versione avanza.

Client A: GETV course -> OK ads version=0

Client B: GETV course -> OK ads version=0

Client A: CAS course 0 v1 -> OK version=1

Client B: CAS course 0 v2 -> ERR version_mismatch current=1

Questo errore non e' rumore di rete. E' il contratto che dice:

- la condizione dichiarata dal client non e' piu' vera;
- quindi la scrittura non e' autorizzata.

```
_, current_version = self._data.get(key, ("", -1))  
next_version = current_version + 1  
self._data[key] = (value, next_version)
```

- una chiave nuova parte implicitamente da versione -1;
- il primo SET produce versione 0;
- ogni aggiornamento incrementa monotonicamente il contatore locale.

Qui emerge già' una scelta di contratto: la versione e' per-chiave, non globale all'intero store.

```
record = self._data.get(key)
current_version = -1 if record is None else record[1]
if current_version != expected_version:
    return f"ERR version_mismatch current={current_version}"
next_version = current_version + 1
self._data[key] = (value, next_version)
```

La logica importante e' tutta dentro una sezione critica:

- confronto della versione;
- decisione di accettare o rifiutare;
- eventuale aggiornamento.

Con GETV e CAS vogliamo ottenere almeno queste proprietà:

- una CAS non può sovrascrivere silenziosamente un update già avvenuto;
- un client con osservazione stantia viene respinto esplicitamente;
- il server produce una storia locale coerente delle versioni di una chiave.

Non stiamo ancora garantendo serializzabilità globale. Stiamo però rendendo osservabile il conflitto.

Il contratto migliora la safety, ma introduce un costo:

- alcune scritture ora possono fallire legittimamente;
- il client deve gestire retry o nuova lettura;
- il sistema puo' progredire solo se i client rispettano il protocollo.

Questa e' una tensione classica:

- piu' informazione semantica all'interfaccia;
- meno comodita' per il chiamante;
- maggiore controllo sui conflitti.

Ambiguità' Da Evitare Nel Contratto

Se esponiamo una versione, dobbiamo specificare bene:

- la versione e' per chiave o globale?
- DELETE cambia la storia della chiave o la resetta?
- quale versione corrisponde a una chiave assente?
- il valore di `current=` in un mismatch e' informativo o normativo?

Un contratto vago qui genera client fragili e implementazioni incompatibili.

Perche' Questa Tappa E' Strategica

CAS e' un ponte tra piu' problemi del corso:

- concorrenza locale;
- replica con conflitti;
- migrazione tra shard;
- coordinamento client-side basato su stato osservato.

Se non fissiamo bene il significato della versione ora, nelle tappe successive ci trasciniamo ambiguita' molto piu' costose.

Con GETV e CAS il server non promette piu' soltanto:

- “questo e' il valore”;
- “questa scrittura e' riuscita”.

Promette anche:

- quale stato il client ha osservato;
- sotto quale preconditione accetta una scrittura;
- come segnala che quella preconditione e' diventata falsa.